

Pods en Podman: teoría y ejercicio integral

Qué es realmente un pod, qué namespaces comparten sus contenedores y cuáles no. Después, un laboratorio con dos contenedores en un mismo pod, cada uno con su volumen, el pod conectado a dos redes, y todo arrancando solo con el sistema operativo mediante Quadlet y systemd.

```
$ podman --version && podman info --format '{{.Host.NetworkBackend}}'
```

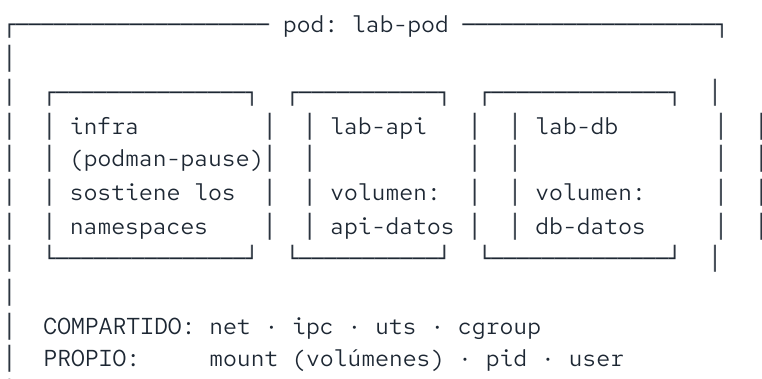
[1. Teoría](#)[2. Qué se comparte](#)[3. Redes](#)[4. Comandos](#)[5. Ejercicio](#)[6. Arranque automático](#)[7. Retos](#)

1. Qué es un pod

Un pod es un grupo de contenedores que comparten un conjunto de namespaces del kernel y se administran como una sola unidad. El concepto viene de Kubernetes y Podman lo implementa de forma nativa: es la razón del nombre, *pod manager*.

Cuando ejecutas `podman pod create`, Podman levanta un contenedor oculto que sostiene los namespaces compartidos. Ese es el **contenedor de infraestructura**, basado en la imagen `localhost/podman-pause` que Podman construye localmente a partir del binario `catatonit`.

Su único trabajo es dormir. Mientras exista, los namespaces del pod siguen vivos aunque todos los contenedores de aplicación se detengan o se reemplacen. Por eso puedes destruir y recrear un contenedor sin que el pod pierda su dirección IP ni sus puertos publicados.



| puertos publicados: siempre a nivel de pod
▼
host RHEL

Por qué agrupar en lugar de usar contenedores sueltos

- **Ciclo de vida conjunto.** `podman pod stop` detiene todo el conjunto y `podman pod rm -f` lo elimina completo.
- **Comunicación por localhost.** Un sidecar habla con la aplicación en `127.0.0.1` sin exponer nada a la red.
- **Límites agregados.** Con `--cpus` y `--memory` a nivel de pod, el cgroup padre limita al conjunto y no a cada pieza por separado.
- **Puente hacia Kubernetes.** `podman kube generate` convierte un pod en manifiesto YAML y `podman kube play` hace lo inverso.

Casos de uso típicos

- Aplicación más proxy inverso que termina TLS.
- Aplicación más agente de logs o de métricas que lee un directorio.
- Base de datos más contenedor de respaldo que se conecta por localhost.
- Contenedor de inicialización que prepara datos antes de que arranque el principal.

2. Qué comparten y qué no

Esta tabla es el corazón de la teoría. Casi todos los errores al armar pods vienen de suponer que se comparte algo que en realidad es privado de cada contenedor.

Namespace	¿Compartido?	Consecuencia práctica
net	Sí	Una sola IP y una sola tabla de puertos para todo el pod. Se hablan por <code>127.0.0.1</code> .
ipc	Sí	Memoria compartida y semáforos POSIX visibles entre contenedores.
uts	Sí	Mismo hostname para todos.
cgroup	Sí	Los límites del pod aplican al conjunto.
pid	No	Cada contenedor ve solo sus procesos. Se habilita con <code>--share pid</code> .
mount	No	Cada contenedor monta sus propios volúmenes. Es lo que hace posible el

resultado	No	Cada contenedor monta sus propios volúmenes. Esto que hace posible el ejercicio de esta guía.
user	No	Cada contenedor define su propio USER y mapeo de UID.

El valor por omisión de `--share` es `cgroup,ipc,net,uts`. Puedes reducirlo con `--share=` vacío o ampliarlo con `--share cgroup,ipc,net,uts,pid`.

La regla que se desprende de la tabla

Los **volúmenes son por contenedor** porque el namespace de montaje no se comparte. Las **redes son por pod** porque el namespace de red sí se comparte. Todo el diseño del laboratorio depende de esa asimetría.

3. Redes dentro de un pod

Aquí hay que hacer una precisión importante sobre el planteamiento del ejercicio.

Dos contenedores del mismo pod no pueden tener redes distintas

Comparten el namespace de red, así que comparten interfaces, IP, rutas y puertos. Si pasas `--network` a un contenedor que se une a un pod, Podman responde con un error del tipo *cannot set network namespace when running in a pod*. La red se define una sola vez, al crear el pod.

Lo que sí se puede hacer, y es lo que haremos, son dos variantes útiles:

1. **Un pod con varias redes.** El pod se une a `red-frontend` y `red-backend` al mismo tiempo. Ambos contenedores reciben las dos interfaces, una IP por red, y alcanzan a otros contenedores en cualquiera de las dos. Así se conecta un pod a un segmento público y a uno privado a la vez.
2. **Aislamiento real por contenedor**, que se logra con **dos pods separados**, cada uno en su red. Está en el reto 1 del final.

Otro detalle que sorprende: en un pod **los puertos se publican al crear el pod**, nunca al crear el contenedor. Y como el puerto vive en un namespace único, dos contenedores del mismo pod no pueden escuchar en el mismo puerto; el segundo falla con *address already in use*.

El backend de red por omisión en RHEL 9 es **netavark**, con **aardvark-dns** resolviendo nombres. Eso significa que el nombre del pod es resoluble desde otros contenedores en la misma red definida por el usuario. La red `podman` por omisión no tiene DNS interno; por eso en el laboratorio creamos redes propias.

4. Comandos esenciales

Referencia rápida antes de entrar al laboratorio.

referencia

Copiar

```
# Crear y listar
podman pod create --name mi-pod -p 8080:8080
podman pod ls
podman ps --pod                # contenedores mostrando su pod

# Inspección
podman pod inspect mi-pod
podman pod stats mi-pod
podman pod top mi-pod

# Agregar contenedores
podman run -d --pod mi-pod --name c1 imagen:tag

# Ciclo de vida
podman pod start    mi-pod
podman pod stop     mi-pod
podman pod pause    mi-pod
podman pod restart  mi-pod
podman pod rm -f     mi-pod      # -f elimina también sus contenedores

# Puente con Kubernetes
podman kube generate mi-pod -f mi-pod.yaml
podman kube play mi-pod.yaml
```

5. Ejercicio integral: pod de laboratorio

Un pod con dos contenedores. `lab-api` es un servicio en Python puro con su volumen de bitácora; `lab-db` es PostgreSQL con su volumen de datos. El pod se conecta a dos redes y publica un solo puerto.

Lo que el ejercicio demuestra

- Volúmenes independientes por contenedor dentro del mismo pod.
- El pod unido a dos redes simultáneas, visibles desde ambos contenedores.
- Comunicación entre contenedores por `127.0.0.1` sin exponer la base de datos.
- Persistencia de la bitácora aunque el contenedor se destruya y se vuelva a crear.
- Arranque automático con el sistema operativo.

Paso 0 Estructura de archivos

```
lab-pod/  
├─ Containerfile  
└─ api.py
```

Paso 1 La aplicación

Solo librería estándar. Escribe cada visita en su volumen y prueba la conexión a PostgreSQL por localhost para comprobar que el namespace de red es compartido.

api.py

Copiar

```
from http.server import HTTPServer, BaseHTTPRequestHandler  
import datetime  
import json  
import os  
import socket  
  
BITACORA = "/opt/app-root/src/datos/bitacora.log"  
  
def registrar(ruta):  
    linea = f"{datetime.datetime.now():%Y-%m-%d %H:%M:%S} {ruta}\n"  
    os.makedirs(os.path.dirname(BITACORA), exist_ok=True)  
    with open(BITACORA, "a") as f:  
        f.write(linea)  
  
def probar_db():  
    """La base vive en otro contenedor del mismo pod: se alcanza por localhost."""  
    try:  
        with socket.create_connection(("127.0.0.1", 5432), timeout=2):  
            return "alcanzable en 127.0.0.1:5432"  
    except OSError as e:  
        return f"no alcanzable ({e.__class__.__name__})"  
  
class Handler(BaseHTTPRequestHandler):  
    def do_GET(self):  
        registrar(self.path)  
  
        if self.path == "/health":  
            cuerpo = json.dumps({"status": "ok"}).encode()  
            tipo = "application/json"  
  
            elif self.path == "/bitacora":  
                try:
```

```

    try:
        with open(BITACORA) as f:
            lineas = f.readlines()[-20:]
    except FileNotFoundError:
        lineas = []
    cuerpo = "".join(lineas).encode()
    tipo = "text/plain; charset=utf-8"

else:
    cuerpo = (
        f"<h1>Pod de laboratorio</h1>"
        f"<p>Hostname del pod: {socket.gethostname()}</p>"
        f"<p>Base de datos: {probar_db()}</p>"
        f"<p><a href='/bitacora'>Ver bitacora</a></p>"
    ).encode()
    tipo = "text/html; charset=utf-8"

self.send_response(200)
self.send_header("Content-Type", tipo)
self.send_header("Content-Length", str(len(cuerpo)))
self.end_headers()
self.wfile.write(cuerpo)

def log_message(self, fmt, *args):
    print(fmt % args, flush=True)

if __name__ == "__main__":
    puerto = int(os.environ.get("PORT", 8080))
    print(f"API escuchando en 0.0.0.0:{puerto}", flush=True)
    HTTPServer(("0.0.0.0", puerto), Handler).serve_forever()

```

Containerfile

Copiar

```

FROM registry.access.redhat.com/ubi9/python-311

LABEL description="API de bitacora para el pod de laboratorio"

WORKDIR /opt/app-root/src
COPY --chown=1001:0 api.py .

# El volumen se montara aqui: el directorio debe existir y ser escribible
RUN mkdir -p /opt/app-root/src/datos && chown 1001:0 /opt/app-root/src/datos

ENV PORT=8080 PYTHONUNBUFFERED=1
EXPOSE 8080
USER 1001

CMD ["python", "api.py"]

```

Paso 2 Construir la imagen

terminal

Copiar

```
cd lab-pod
podman build -t lab-api:1.0 .
podman images | grep lab-api
```

Paso 3 Crear las dos redes y los dos volúmenes

terminal

Copiar

```
# Redes definidas por el usuario: traen DNS interno (aardvark-dns)
podman network create red-frontend
podman network create --subnet 10.89.20.0/24 --gateway 10.89.20.1 red-backend
podman network ls

# Un volumen por contenedor
podman volume create api-datos
podman volume create db-datos
podman volume ls
```

Paso 4 Crear el pod unido a ambas redes

El puerto y las redes se declaran aquí, una sola vez, porque pertenecen al namespace compartido.

terminal

Copiar

```
podman pod create \
  --name lab-pod \
  --network red-frontend,red-backend \
  --publish 8080:8080 \
  --hostname laboratorio

podman pod inspect lab-pod --format '{{.Name}} -> {{.InfraContainerID}}'
```

Paso 5 Agregar los contenedores, cada uno con su volumen

terminal

Copiar

```
# Base de datos: su volumen persiste /var/lib/postgresql/data
podman run -d --pod lab-pod --name lab-db \
  -e POSTGRES_PASSWORD='Admin2026!' \
```

```
-e POSTGRES_PASSWORD=Admin12020: \
-e POSTGRES_DB=laboratorio \
-e PGDATA=/var/lib/postgresql/data/pgdata \
-v db-datos:/var/lib/postgresql/data \
docker.io/library/postgres:16
```

API: su propio volumen, en otra ruta y en otro contenedor

```
podman run -d --pod lab-pod --name lab-api \
-v api-datos:/opt/app-root/src/datos \
localhost/lab-api:1.0
```

```
podman ps --pod
```

Esto falla a propósito

`podman run -d --pod lab-pod --network red-backend ...` devuelve error. El contenedor no puede elegir red porque ya heredó la del pod. Vale la pena ejecutarlo en clase para que los alumnos vean el mensaje.

Paso 6 Verificaciones

terminal

Copiar

A) Ambos ven exactamente las mismas interfaces y las mismas IPs

```
podman exec lab-api ip -br addr
```

```
podman exec lab-db ip -br addr
```

B) Mismo hostname: namespace uts compartido

```
podman exec lab-api hostname
```

```
podman exec lab-db hostname
```

C) Volúmenes distintos: cada uno ve solo el suyo

```
podman exec lab-api ls -l /opt/app-root/src/datos
```

```
podman exec lab-db ls /opt/app-root/src/datos # No such file or directory
```

D) La API alcanza la base por localhost, sin publicar el 5432

```
curl -s http://localhost:8080 | grep -i "Base de datos"
```

E) La bitácora se escribe en el volumen de la API

```
curl -s http://localhost:8080/health > /dev/null
```

```
curl -s http://localhost:8080/bitacora
```

F) Los procesos no se ven entre sí: pid no compartido

```
podman exec lab-api ps -ef
```

G) Vista agregada del pod

```
podman pod top lab-pod
```

```
podman pod stats --no-stream lab-pod
```

Paso 7 Probar la persistencia

terminal

Copiar

```
# Destruir solo el contenedor de la API y recrearlo
podman rm -f lab-api
podman run -d --pod lab-pod --name lab-api \
  -v api-datos:/opt/app-root/src/datos \
  localhost/lab-api:1.0

# La bitácora anterior sigue ahí
curl -s http://localhost:8080/bitacora
```

Paso 8 Guardar el diseño como manifiesto

Antes de automatizar, exporta el pod a YAML. Sirve como documentación y como puente hacia Kubernetes.

terminal

Copiar

```
podman kube generate lab-pod -f lab-pod.yaml
head -40 lab-pod.yaml
```

6. Arranque automático con el sistema operativo

La forma vigente en RHEL 9 es **Quadlet**: archivos de unidad declarativos que systemd traduce a servicios al hacer `daemon-reload`. Sustituye a `podman generate systemd`, que está deprecado.

Requisito de versión

Las unidades `.container`, `.volume` y `.network` existen desde Podman 4.4. Las unidades `.pod` requieren **Podman 5.0 o superior**. Verifica con `podman --version`; al final hay una alternativa para versiones anteriores.

6.1 Dónde van los archivos

Modo	Directorio	Recarga

Rootless	~/.config/containers/systemd/	systemctl --user daemon-reload
Root	/etc/containers/systemd/	sudo systemctl daemon-reload

Usaremos rootless, que es la práctica recomendada. Empieza por limpiar lo creado a mano, porque Quadlet administrará sus propios recursos:

terminalCopiar

```
podman pod rm -f lab-pod
podman volume rm api-datos db-datos
podman network rm red-frontend red-backend
mkdir -p ~/.config/containers/systemd
```

6.2 Las redes

red-frontend.networkCopiar

```
[Unit]
Description=Red frontend del laboratorio

[Network]
NetworkName=red-frontend
```

red-backend.networkCopiar

```
[Unit]
Description=Red backend del laboratorio

[Network]
NetworkName=red-backend
Subnet=10.89.20.0/24
Gateway=10.89.20.1
```

6.3 Los volúmenes

api-datos.volumeCopiar

```
[Unit]
Description=Volumen de bitacora de la API

[Volume]
VolumeName=api-datos
```

db-datos.volume

Copiar

```
[Unit]
Description=Volumen de datos de PostgreSQL
```

```
[Volume]
VolumeName=db-datos
```

6.4 El pod

Aquí viven las redes y la publicación de puertos, igual que en la línea de comandos.

laboratorio.pod

Copiar

```
[Unit]
Description=Pod de laboratorio (API + PostgreSQL)
```

```
[Pod]
PodName=lab-pod
Network=red-frontend.network
Network=red-backend.network
PublishPort=8080:8080
PodmanArgs=--hostname laboratorio
```

```
[Install]
WantedBy=default.target
```

6.5 Los contenedores

Cada uno declara su propio volumen y apunta al pod con `Pod=` . Ninguno declara red ni puertos: los hereda.

lab-db.container

Copiar

```
[Unit]
Description=PostgreSQL del laboratorio
```

```
[Container]
ContainerName=lab-db
Image=docker.io/library/postgres:16
Pod=laboratorio.pod
Volume=db-datos.volume:/var/lib/postgresql/data
Environment=POSTGRES_DB=laboratorio
Environment=PGDATA=/var/lib/postgresql/data/pgdata
```

```
Environment=PGDATA=/var/lib/postgresql/data/pgdata  
Secret=pgpass,type=env,target=POSTGRES_PASSWORD
```

```
[Service]  
Restart=always  
TimeoutStartSec=300
```

```
[Install]  
WantedBy=default.target
```

lab-api.container

[Copiar](#)

```
[Unit]  
Description=API de bitacora del laboratorio  
Requires=lab-db.service  
After=lab-db.service  
  
[Container]  
ContainerName=lab-api  
Image=localhost/lab-api:1.0  
Pod=laboratorio.pod  
Volume=api-datos.volume:/opt/app-root/src/datos  
Environment=PORT=8080  
  
[Service]  
Restart=always  
TimeoutStartSec=300  
  
[Install]  
WantedBy=default.target
```

6.6 El secreto de la contraseña

La unidad de la base referencia un secreto de Podman en lugar de escribir la contraseña en el archivo.

terminal

[Copiar](#)

```
printf 'Admin2026!' | podman secret create pgpass -  
podman secret ls
```

6.7 Activar y probar

terminal

[Copiar](#)

```
# Permite que los servicios del usuario corran sin sesión iniciada  
loginctl enable-linger $USER
```

```
systemctl enable tinger@OSER
```

```
# Genera los servicios a partir de los archivos Quadlet
systemctl --user daemon-reload

# Arrancar: el pod y sus dependencias suben solos
systemctl --user start lab-api.service

# Estado
systemctl --user status laboratorio-pod.service lab-db.service lab-api.service
podman ps --pod

# Prueba funcional
curl -s http://localhost:8080
```

No uses systemctl enable con Quadlet

Las unidades son generadas, no existen en disco como servicios, así que `systemctl --user enable lab-api.service` falla con *unit file does not exist*. La sección `[Install]` `WantedBy=default.target` es la que las hace arrancar al inicio. Ojo con el nombre del servicio del pod: el archivo `laboratorio.pod` genera `laboratorio-pod.service`.

6.8 Validar el arranque real

terminal

Copiar

```
sudo reboot

# Al volver, sin iniciar sesión gráfica ni ejecutar nada:
podman ps --pod
curl -s http://localhost:8080/bitacora
journalctl --user -u lab-api.service --since "10 min ago"

# Depurar los archivos Quadlet si algo no aparece
/usr/libexec/podman/quadlet -user -dryrun
```

Alternativa para Podman 4.x, sin unidades .pod

Si tu RHEL trae Podman 4.9, arma el pod a mano y genera las unidades con el método antiguo, sabiendo que está deprecado.

terminal

Copiar

```
# Con el pod ya armado como en la sección 5:
mkdir -p ~/.config/systemd/user && cd ~/.config/systemd/user
```

```
podman generate systemd --new --files --name lab-pod
systemctl --user daemon-reload
systemctl --user enable --now pod-lab-pod.service
loginctl enable-linger $USER
```

Notas de operación en RHEL

- Abre el puerto si accedes desde otra máquina: `sudo firewall-cmd --add-port=8080/tcp --permanent && sudo firewall-cmd --reload`.
- En rootless no puedes publicar puertos menores a 1024 sin ajustar `net.ipv4.ip_unprivileged_port_start`.
- Con SELinux en *enforcing* los volúmenes con nombre se etiquetan solos; si usas bind mounts del host agrega `:Z`.
- Para actualización automática de imagen agrega `AutoUpdate=registry` en `[Container]` y habilita `podman-auto-update.timer`.

7. Retos para cerrar

Tres extensiones que consolidan la teoría de la sección 2.

1. **Aislamiento real por red.** Rehaz el laboratorio con *dos pods*: `pod-front` solo en `red-frontend` y `pod-data` solo en `red-backend`. Comprueba que no se ven. Después conecta `pod-front` también a `red-backend` con `podman network connect` y verifica que ahora resuelve `pod-data` por nombre gracias a `aardvark-dns`. Discute la diferencia con la solución de un solo pod.
2. **Sidecar con volumen compartido.** Agrega un tercer contenedor que monte `api-datos` en modo lectura (`:ro`) y ejecute `tail -F` sobre la bitácora. Demuestra que un volumen sí puede compartirse entre contenedores, pero solo si ambos lo montan explícitamente: no se hereda por estar en el mismo pod.
3. **Del pod a Kubernetes.** Toma el `lab-pod.yaml` del paso 8, identifica cómo quedaron representados los volúmenes y explica por qué la configuración de redes no aparece tal cual en un manifiesto de Kubernetes.

Limpieza

terminal

Copiar

```
systemctl --user stop lab-api.service lab-db.service laboratorio-pod.service
rm -f ~/.config/containers/systemd/{laboratorio.pod,lab-api.container,lab-db.container}
rm -f ~/.config/containers/systemd/{api-datos.volume,db-datos.volume}
```

```
rm -f ~/.config/containers/systemd/{red-frontend.network,red-backend.network}
systemctl --user daemon-reload

podman pod rm -f lab-pod
podman volume rm api-datos db-datos
podman network rm red-frontend red-backend
podman secret rm pgpass
podman image prune -f
```

Sintaxis para Podman 5.x sobre RHEL 9 y derivados. Las unidades .pod requieren Podman 5.0 o superior.